



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Rosas Martinez Francisco Javier

N° de Cuenta: 320109766

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 25/Octubre/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1:

Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del cofre de su coche y al abrir y cerrar el cofre ilumine en esa dirección.

Para resolver el ejercicio, tuve que utilizar de nueva cuenta el método *SetPos()* para mi luz, con el fin de actualizar su posición en tiempo real. Posteriormente, implementé una luz de color amarillo que seguiría la dirección del cofre. Además, añadí el movimiento del cofre, asignando las teclas 1 y 2 para controlarlo, y limitando su apertura a un ángulo máximo de 90°.

Por último, para determinar correctamente la posición de la luz, multipliqué la matriz del modelo por un vec4 con un valor definido en el eje x, lo que permitió establecer la dirección adecuada. Gracias a esta operación, logré que la luz siguiera de manera precisa el movimiento de apertura del cofre.

Bloques de código generado

Window.h

```
GLfloat getrotar() { return rotacion; }
```

```
GLfloat rotacion;
```

Window.cpp

```
//cuando se preciona 1 o 2 rota
if (key == GLFW_KEY_1)
{
    //Para verificar que solo se mueva de 0° a 90°
    if (theWindow->rotacion < 90.0f) {
        theWindow->rotacion += 2.0;
    }
}
if (key == GLFW_KEY_2)
{
    if (theWindow->rotacion > 2.0f) {
        theWindow->rotacion -= 2.0;
    }
}
```

P08-320109766.cpp

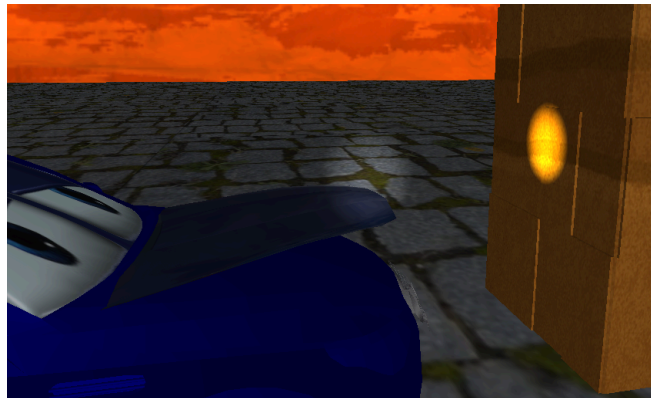
```
// Ejercicio 1
spotLights[1] = SpotLight(1.0f, 1.0f, 0.0f,
    1.0f, 2.0f,
    10.0f, 10.0f, 5.0f,
    1.0f, 0.0f, 0.0f,
    1.0f, 0.0f, 0.01f,
    15.0f);
spotLightCount++;
```

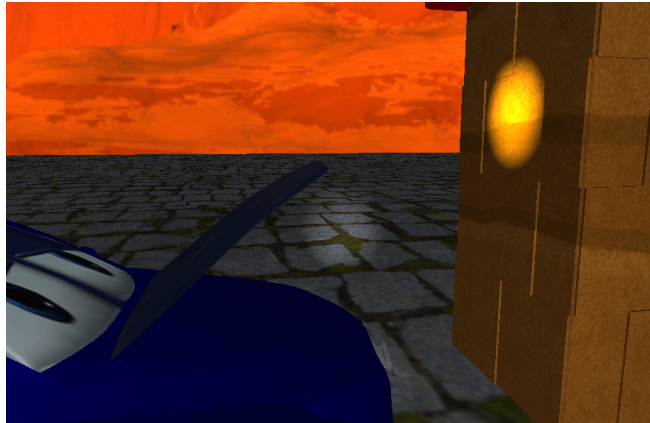
```
// Cofre
model = modelaux;
model = glm::translate(model, glm::vec3(5.4f, 1.45f, 0.8f));
model = glm::rotate(model, glm::radians(0.0f + mainWindow.getrotar()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
cofre.RenderModel();

// Posición de la luz en el cofre
spotLights[1].SetPos(glm::vec3(
    model * glm::vec4(3.0f, 0.0f, 0.0f, 1.0f)
));
```

Código en ejecución





Ejercicio 2:

Agregar luz de tipo spotlight para el coche de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight hacia adelante y al retroceder ((mover con teclado hacia dirección de X positiva) ilumine con un spotlight hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán de acuerdo a alguna bandera asignada por ustedes.

Para resolver el ejercicio, fue necesario crear dos arreglos distintos de luces tipo SpotLight. En el primer arreglo coloqué como último elemento la luz delantera, mientras que en el segundo incluí la luz trasera. De esta manera, mediante las teclas U y Y se podía alternar entre un arreglo y otro. Estas mismas teclas controlaban el movimiento del coche, logrando así que, al avanzar, se encendiera la luz delantera, y al retroceder, se activara la luz trasera.

Bloques de código generado

Window.h

```
GLfloat getlucesspot() { return lucesspot; }  
bool getShouldClose() {
```

```
    default lucesspot;  
    GLfloat lucesspot;
```

Window.cpp

```
if (key == GLFW_KEY_Y)
{
    theWindow->lucesspot = 0.0;
}

if (key == GLFW_KEY_U)
{
    theWindow->lucesspot = 1.0;
}
```

P08-320109766.cpp

```
// Ejercicio 2 luz de adelante del coche
spotLights[2] = SpotLight(1.0f, 1.0f, 1.0f,
    1.0f, 2.0f,
    10.0f, 10.0f, 5.0f,
    1.0f, 0.0f, 0.0f,
    1.0f, 0.0f, 0.01f,
    25.0f);
spotLightCount++;

// Arreglo 2 para la luz trasera
spotLights2[0] = spotLights[0];
spotLights2[1] = spotLights[1];
```

```
// Ejercicio 2 luz de atras del coche
spotLights2[2] = SpotLight(1.0f, 0.0f, 0.0f,
    1.0f, 2.0f,
    10.0f, 10.0f, 5.0f,
    -1.0f, 0.0f, 0.0f,
    1.0f, 0.0f, 0.01f,
    25.0f);
```

```
// luz deantera
if (mainWindow.getlucesspot() < 0.5f) {
    shaderList[0].SetSpotLights(spotLights, spotLightCount);
    //printf("primer arreglo de luces\n");
}
else { // luz trasera
    shaderList[0].SetSpotLights(spotLights2, spotLightCount);
    //printf("segundo arreglo de luces\n");
}
```

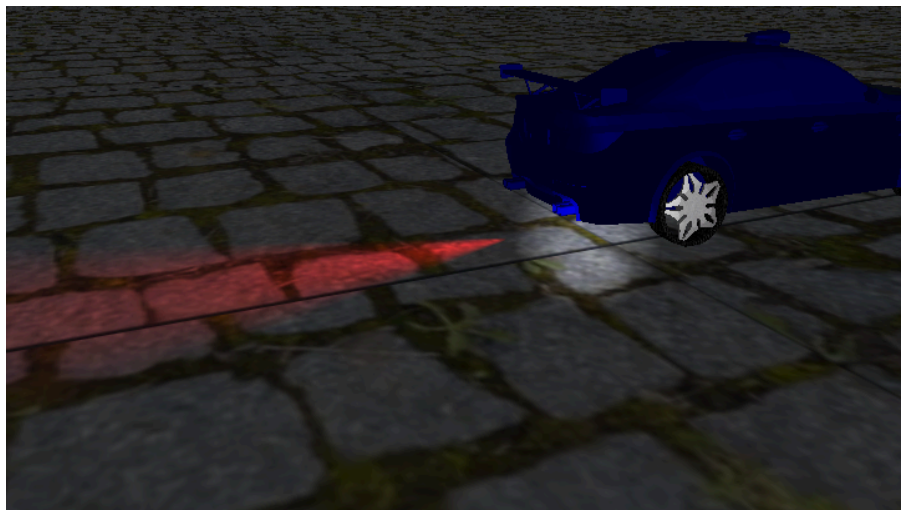
```

// Coche base
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 2.30f, -0.5f));
model = glm::translate(model, glm::vec3(0.0f + mainWindow.getmuevex(), 0.5f, -3.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
coche.RenderModel();

// Para que la luz tenga la posición del coche
spotLights[2].SetPos(glm::vec3(modelaux[3]) + glm::vec3(10.0f, -4.0f, 0.6f));
spotLights[2].SetPos(glm::vec3(modelaux[3]) + glm::vec3(-12.0f, -4.0f, 0.6f));

```

Código en ejecución



Ejercicio 3:

Agregar otra luz de tipo puntual ligada a un modelo elegido por ustedes (no lámpara) y que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara como la luz de este modelo (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual).

Con este ejercicio, tuve que modificar mi arreglo de luces puntuales, dejando únicamente dos luces: una luz genérica como primer elemento y la luz correspondiente a mi modelo. Posteriormente, agregué el modelo en el main y coloqué la luz en una posición adecuada. Finalmente, asigné la tecla P para controlar el encendido y apagado de la luz.

Bloques de código generado

Window.h

```
GLboolean getPrendida() { return luzprendida; }
```

```
GLboolean luzprendida;
```

Window.cpp

```
if (key == GLFW_KEY_P && action == GLFW_PRESS)
{
    theWindow->luzprendida = !theWindow->luzprendida;
}
```

P08-320109766.cpp

```
Model cactus;
```

```
cactus = Model();
cactus.LoadModel("Models/cactus.obj");
```

```

//Declaración de primer arreglo de luces puntuales
//verde
pointLights[0] = PointLight(0.0f, 1.0f, 0.0f,
    0.01f, 0.01f,
    0.0f, -1.0f, 10.0f,
    0.3f, 0.3f, 0.3f);
pointLightCount++;

// Ejercicio 3
//// luz cactus
pointLights[1] = PointLight(0.0f, 1.0f, 0.0f,
    1.0f, 1.0f,
    0.0f, 2.0f, 10.0f,
    1.0f, 0.0f, 0.2f);
pointLightCount++;

```

```

// Luz prendida
if (mainWindow.getPrendia()) {
    shaderList[0].SetPointLights(pointLights, pointLightCount);
    printf("primer arreglo de luces\n");
}
else { // Luz apagada
    shaderList[0].SetPointLights(pointLights, pointLightCount-1);
    printf("segundo arreglo de luces\n");
}

```

Código en ejecución





2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

- Con esta práctica afortunadamente no tuve inconvenientes.

Conclusión:

Con estos ejercicios aprendí a manejar mejor las luces dentro de OpenGL. Logré entender cómo cambiar su posición, dirección, color y atenuación, además de hacer que siguieran el movimiento de los modelos. También logré controlar las luces con el teclado, encendiéndolas o cambiándolas según el movimiento del coche o del cofre. En general, esta práctica me ayudó a comprender y reforzar cómo funcionan las luces en tiempo real y cómo hacer que se vean más realistas dentro de mi proyecto.

Bibliografía :

OpenGL Architecture Review Board, Shreiner, D., Woo, M., Neider, J., & Davis, T. (2010). OpenGL Programming Guide: The Official Guide to Learning OpenGL (7th ed.). Addison-Wesley.

OpenGL. (s. f.). OpenGL Documentation. Khronos Group. Recuperado de <https://www.khronos.org/opengl/>

Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3 (8th ed.). Addison-Wesley

Professional.